School of Computation, Information and Technology
Technische Universität München

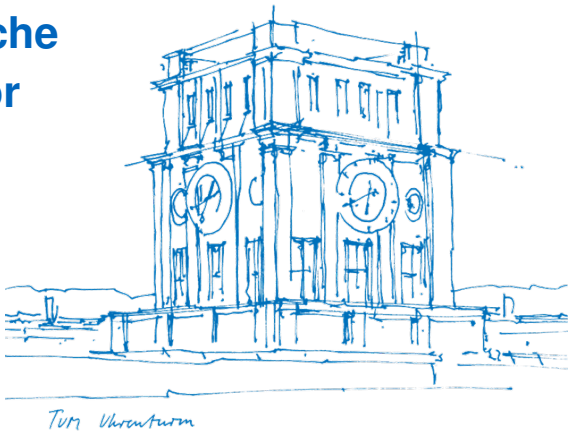
Übung 08: Maschinensprache und Single-Cycle-Prozessor

Einführung in die Rechnerarchitektur

Niklas Ladurner

School of Computation, Information and Technology
Technische Universität München

5. Dezember 2025



Feedback

t1p.de/era2526



home.in.tum.de/~ladu/



Keine Garantie für die Richtigkeit der Tutorfolien.

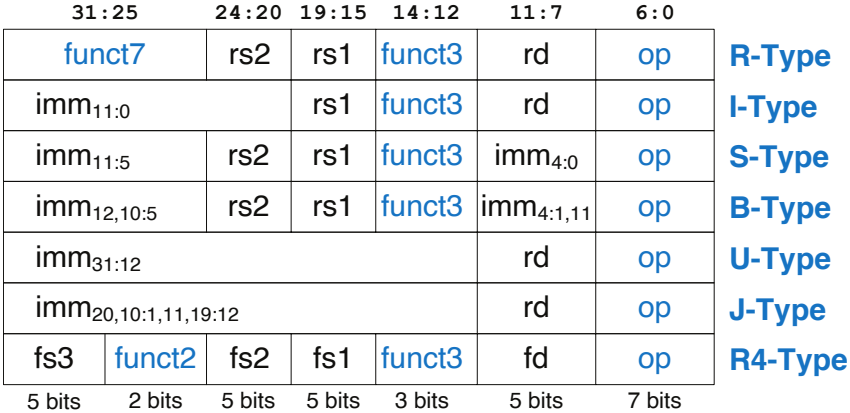
Bei Unklarheiten/Unstimmigkeiten haben VL/ZÜ-Folien recht!

Instruktionstypen[1]

R	Register-Register-Operationen	`add`, `sub`, `sll`
I	Short Immediates (12 Bit) und Ladebefehle	`jalr`, `lw`, `ori`
S	Speicherbefehle	`sw`, `sh`
B	Branches (bedingte Sprünge)	`beq`, `blt`, `bgtu`
U	Long Immediates (20 Bit)	`lui`, `auipc`
J	Jumps (unbedingte Sprüge) mit Long Immediate	`jal`
R4	Floating-Point-Operationen, für ERA nicht relevant	

[1] NB: Die Befehle für die einzelnen Typen sind nur auszugsweise angegeben.

Assemblierung



(Quelle: Vorlesungsmaterialien ERA)

Assemblierung: Beispiel

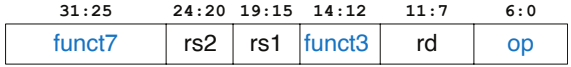
```
xor t2, t1, t0
```

1. Instruktionstyp feststellen

Assemblierung: Beispiel

```
xor t2, t1, t0
```

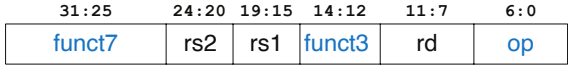
1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden



Assemblierung: Beispiel

```
xor t2, t1, t0
```

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden

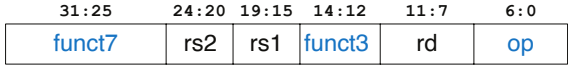


op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen

```
xor t2, t1, t0
```

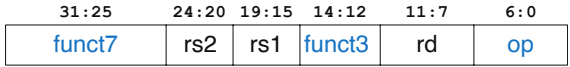


op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen
5. Registermapping:
   (`zero` $\mapsto$ `x0`, ..., `t6` $\mapsto$ `x31`)

`xor t2, t1, t0`



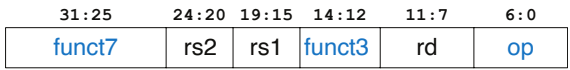
op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

`t2` $\mapsto$ `x7`, `t1` $\mapsto$ `x6`, `t0` $\mapsto$ `x5`

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen
5. Registermapping:
   (`zero` $\mapsto$ `x0`, ..., `t6` $\mapsto$ `x31`)
6. Binärzahl zusammenbauen

`xor t2, t1, t0`



op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

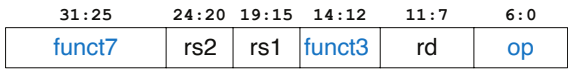
`t2` $\mapsto$ `x7`, `t1` $\mapsto$ `x6`, `t0` $\mapsto$ `x5`

`funct7 rs2 rs1 funct3 rd op`

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen
5. Registermapping:
   (`zero` $\mapsto$ `x0`, ..., `t6` $\mapsto$ `x31`)
6. Binärzahl zusammenbauen

`xor t2, t1, t0`



op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

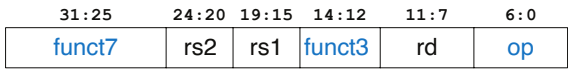
`t2` $\mapsto$ `x7`, `t1` $\mapsto$ `x6`, `t0` $\mapsto$ `x5`

`funct7 rs2 rs1 funct3 rd op`

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen
5. Registermapping:
   (`zero` $\mapsto$ `x0`, ..., `t6` $\mapsto$ `x31`)
6. Binärzahl zusammenbauen

`xor t2, t1, t0`



op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

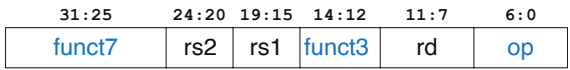
`t2` $\mapsto$ `x7`, `t1` $\mapsto$ `x6`, `t0` $\mapsto$ `x5`

`0000000 rs2 rs1 100 rd 0110011`

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen
5. Registermapping:
   (`zero` $\mapsto$ `x0`, ..., `t6` $\mapsto$ `x31`)
6. Binärzahl zusammenbauen

`xor t2, t1, t0`



op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

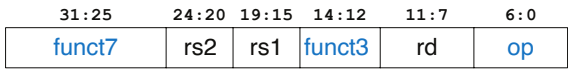
`t2` $\mapsto$ `x7`, `t1` $\mapsto$ `x6`, `t0` $\mapsto$ `x5`

`0000000 rs2 rs1 100 rd 0110011`

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen
5. Registermapping:
   (`zero` $\mapsto$ `x0`, ..., `t6` $\mapsto$ `x31`)
6. Binärzahl zusammenbauen

`xor t2, t1, t0`



op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

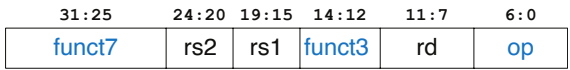
`t2` $\mapsto$ `x7`, `t1` $\mapsto$ `x6`, `t0` $\mapsto$ `x5`

0000000 rs2 rs1 100 rd 0110011

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen
5. Registermapping:
   (`zero` $\mapsto$ `x0`, ..., `t6` $\mapsto$ `x31`)
6. Binärzahl zusammenbauen

`xor t2, t1, t0`



op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

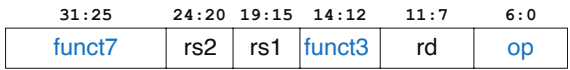
`t2` $\mapsto$ `x7`, `t1` $\mapsto$ `x6`, `t0` $\mapsto$ `x5`

0000000 00101 00110 100 00111 0110011

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen
5. Registermapping:
   (`zero` $\mapsto$ `x0`, ..., `t6` $\mapsto$ `x31`)
6. Binärzahl zusammenbauen

`xor t2, t1, t0`



op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

`t2` $\mapsto$ `x7`, `t1` $\mapsto$ `x6`, `t0` $\mapsto$ `x5`

`0000000 00101 00110 100 00111 0110011`

Assemblierung: Beispiel

1. Instruktionstyp feststellen $\rightarrow$ R
2. Dazugehöriges Layout in Tabelle finden
3. Instruktion in Tabelle finden
4. Instruktionsspezifische Werte ablesen
5. Registermapping:
   (`zero` $\mapsto$ `x0`, ..., `t6` $\mapsto$ `x31`)
6. Binärzahl zusammenbauen

`xor t2, t1, t0`

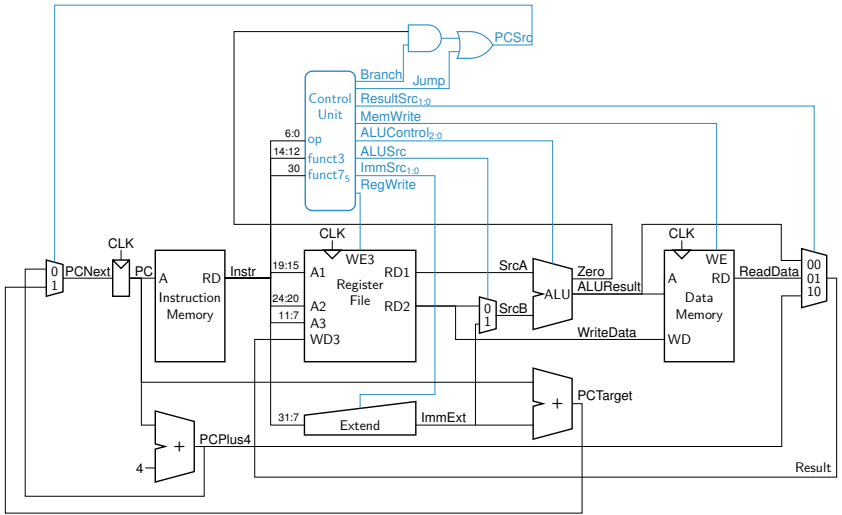
31:25	24:20	19:15	14:12	11:7	6:0
funct7	rs2	rs1	funct3	rd	op

op	funct3	funct7	Type	Instruction
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`

`t2` $\mapsto$ `x7`, `t1` $\mapsto$ `x6`, `t0` $\mapsto$ `x5`

`0b 0000 0000 0101 0011 0100 0011 1011 0011` =
`0x005343B3`

RISC-V Single-Cycle-Prozessor



Fragen?

Links

- Zulip: „ERA Tutorium – Mi-1600-3“ bzw. „ERA Tutorium – Fr-1500-1“
- ERA-Moodle-Kurs
- ERA-Artemis-Kurs
- Prozessor-Assets (kein offizielles Material!)
- RISC-V Assembler

School of Computation, Information and Technology
Technische Universität München

Übung 08: Maschinensprache und Single-Cycle-Prozessor

Einführung in die Rechnerarchitektur

Niklas Ladurner

School of Computation, Information and Technology
Technische Universität München

5. Dezember 2025

